

INTRANET DE ESTUDOS

Documentação Técnica e Acadêmica

Desenvolvimento de Sistemas — 1º Semestre
Integração das Disciplinas: Algoritmos, Programação Estruturada,
Aplicativos Computacionais, Comunicação Organizacional e Ética

1. Introdução

O presente documento descreve o funcionamento técnico e acadêmico do projeto Intranet de Estudos, desenvolvido como atividade integradora do primeiro semestre do curso de Desenvolvimento de Sistemas. O projeto tem como objetivo unificar em uma única plataforma digital os conteúdos, ferramentas e funcionalidades que apoiam o aprendizado das cinco disciplinas do semestre.

A motivação central do projeto é demonstrar, na prática, como os conceitos estudados em cada disciplina se articulam: a lógica de programação de Algoritmos e Programação Estruturada, o domínio de ferramentas digitais de Aplicativos Computacionais, os princípios de comunicação eficaz de Comunicação Organizacional, e os valores éticos de Ética e Responsabilidade Social.

1.1 Objetivos

- Desenvolver uma aplicação web funcional integrando frontend e backend
 - Aplicar os conceitos de algoritmos e programação estruturada em um projeto real
 - Implementar um sistema de recomendação de conteúdo personalizado por usuário
 - Praticar a comunicação organizacional por meio de um sistema de notícias
 - Refletir sobre ética no desenvolvimento de software e uso de dados pessoais
-

2. Arquitetura do Sistema

O sistema adota a arquitetura cliente-servidor, padrão amplamente utilizado no desenvolvimento de aplicações web. Nessa abordagem, as responsabilidades são divididas em três camadas distintas, cada uma com função específica e bem delimitada.

2.1 Camadas da Aplicação

2.1.1 Camada de Apresentação (Frontend)

Corresponde à interface com a qual o usuário interage diretamente. É composta por arquivos HTML, CSS e JavaScript localizados na pasta static/. Não acessa os dados diretamente — toda comunicação com o servidor ocorre por meio de requisições HTTP assíncronas utilizando a API Fetch do JavaScript.

2.1.2 Camada de Lógica de Negócio (Backend)

Implementada em Python com o microframework Flask, é responsável por processar as requisições recebidas do frontend, aplicar as regras de negócio —

como autenticação e recomendação — e retornar respostas no formato JSON. Composta pelos arquivos `app.py`, `routes.py`, `database.py` e `recomendacoes.py`.

2.1.3 Camada de Dados

Responsável pela persistência das informações do sistema. Nesta versão do projeto, os dados são armazenados em arquivos no formato JSON, localizados na pasta `data/`. Essa abordagem simplifica o aprendizado e permite evolução futura para sistemas de banco de dados relacionais como SQLite ou PostgreSQL.

2.2 Estrutura de Arquivos

Arquivo	Camada	Responsabilidade
<code>app.py</code>	Backend	Ponto de entrada; inicializa o servidor Flask e registra as rotas
<code>routes.py</code>	Backend	Define os endpoints da API REST e as regras de acesso
<code>recomendacoes.py</code>	Backend	Algoritmo de recomendação personalizada de notícias
<code>database.py</code>	Backend	Camada de acesso aos dados; leitura e escrita dos arquivos JSON
<code>data/usuarios.json</code>	Dados	Base de dados de usuários, credenciais e interesses
<code>data/noticias.json</code>	Dados	Repositório de notícias cadastradas no sistema
<code>data/disciplinas.json</code>	Dados	Conteúdos, materiais e quizzes das disciplinas
<code>static/login.html</code>	Frontend	Interface de autenticação do usuário
<code>static/dashboard.html</code>	Frontend	Painel principal com notícias recomendadas
<code>static/disciplinas.html</code>	Frontend	Módulo de estudo com quizzes e anotações
<code>static/noticias.html</code>	Frontend	Gerenciamento de notícias com criação e exclusão

3. Backend — Python e Flask

O backend é o núcleo do sistema. É responsável por receber as requisições do navegador, processá-las e devolver as respostas. O Flask foi escolhido por sua simplicidade e aderência ao nível de conhecimento do primeiro semestre — ele permite criar um servidor web completo com poucas linhas de código.

3.1 Inicialização do Servidor

O arquivo `app.py` inicializa a aplicação Flask, registra o conjunto de rotas definidas em `routes.py` por meio do mecanismo de Blueprint, e define a chave secreta utilizada para criptografar as sessões de usuário. Ao ser executado, o servidor passa a escutar requisições na porta 5000 do endereço local.

3.2 API REST

O sistema implementa uma API REST (Representational State Transfer), padrão arquitetural que organiza a comunicação entre cliente e servidor por meio de métodos HTTP semânticos. Cada endpoint corresponde a um recurso do sistema e responde em formato JSON.

Método	Endpoint	Função
POST	/api/login	Autentica o usuário e inicia uma sessão
POST	/api/logout	Encerra a sessão do usuário
GET	/api/me	Retorna os dados do usuário autenticado
GET	/api/recomendacoes	Executa o algoritmo e retorna notícias recomendadas
GET	/api/noticias	Lista todas as notícias cadastradas
POST	/api/noticias	Cria uma nova notícia
DELETE	/api/noticias/<id>	Remove uma notícia pelo identificador
GET	/api/disciplinas	Retorna os dados de todas as disciplinas

3.3 Autenticação e Sessões

O sistema de autenticação verifica as credenciais do usuário (e-mail e senha) consultando o arquivo `usuarios.json`. Quando válidas, o identificador do usuário é armazenado na sessão do Flask — um mecanismo que mantém o estado entre requisições por meio de um cookie criptografado enviado ao navegador. Rotas protegidas verificam a existência dessa sessão antes de processar qualquer dado.

4. Algoritmo de Recomendação

O algoritmo de recomendação é o componente central do projeto e tem origem direta no código desenvolvido na disciplina de Algoritmos e Programação Estruturada. Sua função é filtrar, dentre todas as notícias disponíveis, aquelas mais relevantes para cada usuário.

4.1 Lógica de Funcionamento

O algoritmo aplica duas regras de filtragem para cada notícia do repositório:

- Regra 1 — Correspondência de unidade: verifica se a unidade organizacional do usuário está na lista de unidades-alvo da notícia
- Regra 2 — Interseção de interesses: verifica se há elementos em comum entre os interesses cadastrados do usuário e as tags associadas à notícia

Uma notícia é recomendada quando ao menos uma das regras é satisfeita. O algoritmo utiliza a operação de interseção de conjuntos (set & set) do Python para identificar os interesses em comum, registrando-os como motivo da recomendação exibida ao usuário.

4.2 Evolução do Código

Na versão original, desenvolvida na disciplina de Algoritmos, o algoritmo imprimia os resultados diretamente no terminal por meio da função `print()`. Na versão integrada ao sistema web, a única alteração foi substituir a saída pelo terminal pelo retorno de um dicionário Python, que o Flask converte automaticamente para JSON e transmite ao navegador. A lógica de filtragem permanece idêntica, demonstrando o princípio da reutilização de código.

5. Frontend — HTML, CSS e JavaScript

O frontend é a camada visual do sistema. Cada página é um arquivo HTML independente que utiliza CSS para estilização e JavaScript para comportamento dinâmico e comunicação com o backend.

5.1 Comunicação com o Backend

Toda a comunicação entre frontend e backend ocorre por meio da API Fetch, uma interface nativa do JavaScript moderno que permite realizar requisições HTTP de forma assíncrona. O uso da palavra-chave `await` suspende a execução da função até que a resposta do servidor seja recebida, sem bloquear a interface do usuário.

A opção `credentials: 'include'` é obrigatória em todas as requisições para que o cookie de sessão seja enviado ao servidor, permitindo que o Flask identifique o usuário autenticado.

5.2 Módulos da Interface

- Login: formulário de autenticação que envia as credenciais via POST e redireciona ao dashboard em caso de sucesso

- Dashboard: carrega e exibe as notícias recomendadas pelo algoritmo de recomendação, identificando o usuário pelo nome
 - Disciplinas: apresenta os conteúdos, materiais de estudo e quizzes interativos de cada disciplina, com anotações pessoais salvas localmente
 - Notícias: permite criar e excluir notícias, com atualização imediata da lista sem recarregar a página
-

6. Integração com as Disciplinas

O projeto foi concebido como instrumento de integração curricular. Cada componente do sistema estabelece correspondência direta com conteúdos estudados ao longo do semestre.

Algoritmos e Programação Estruturada

O algoritmo de recomendação em `recomendacoes.py` aplica diretamente os conceitos de variáveis, estruturas condicionais, laços de repetição, funções e operações com conjuntos. O módulo `database.py` demonstra a leitura e escrita de arquivos, tema de Programação Estruturada.

Aplicativos Computacionais

O domínio do ambiente de desenvolvimento (VSCode), do terminal de comandos, do gerenciador de pacotes `pip` e dos formatos de arquivo JSON constituem competências diretamente relacionadas à disciplina de Aplicativos Computacionais.

Comunicação Organizacional

O sistema de notícias com segmentação por unidade e interesses simula um canal de comunicação interna corporativa, aplicando os princípios de comunicação dirigida e relevância da mensagem para o receptor estudados na disciplina.

Ética e Responsabilidade Social

O sistema coleta e processa dados pessoais dos usuários — e-mail, unidade e interesses. Isso suscita reflexões sobre privacidade de dados, consentimento informado e responsabilidade no tratamento de informações pessoais, temas centrais da disciplina de Ética e Responsabilidade Social e da legislação brasileira (LGPD — Lei nº 13.709/2018).

7. Considerações Finais

O projeto Intranet de Estudos demonstra que os conceitos fundamentais de programação, quando aplicados a um problema real e significativo, ganham concretude e sentido. A trajetória de um algoritmo que imprimia resultados no terminal até uma aplicação web completa, com autenticação, API REST e interface visual, ilustra o processo de evolução incremental característico do desenvolvimento de software profissional.

O sistema está estruturado para crescer ao longo do curso: novas disciplinas podem ser adicionadas ao arquivo disciplinas.json sem qualquer alteração no código; novas funcionalidades podem ser implementadas seguindo os padrões já estabelecidos. Ao final do curso, o projeto constituirá um portfólio técnico que documenta a jornada de aprendizado do estudante.

— *Fim do Documento* —